Relatório final — Detecção de Intrusão em Redes

Trabalho em Competição · Inteligência Artificial · Profa. Cristiane Neri Nobre · Equipe XX

Nov 12, 2026 · @Luisa

Resumo

O modelo final é uma **random forest de 300 árvores**. Na validação cruzada 5×2, feita só no treino, o F1-macro esperado é **0,985** sem o atributo `score_ids_legado` e **0,991** com ele. Três decisões explicam a maior parte do resultado: escolher o limiar fora de cada fold em vez de balancear as classes, imputar `service` e `flag` com uma random forest (97% de acerto, contra 57% da moda) e remover do treino as duplicatas e os pares contraditórios. Nenhuma etapa do pipeline vê o teste, e o notebook `etapa4_final.ipynb` reproduz a submissão.

1. Dados e limpeza (requisito 1)

O treino tem 69.833 conexões, com 5,8% de anomalias, e o teste tem 24.333. A limpeza (`limpeza.py`) usa só regras fixas.

Só no treino, a limpeza remove:

- 11.355 linhas duplicadas;
- 1.702 pares contraditórios (3.404 linhas): mesmos atributos e classes diferentes, sempre um `normal` e um `anomaly` .

Nos dois arquivos, ela:

- padroniza o texto;
- troca 5.859 valores impossíveis por ausentes;
- troca `su_attempted` = 2 por 1;
- remove 6 colunas copiadas ou constantes;
- remove os 2 atributos com vazamento, `tempo_resposta_analista_min` e `tipo_alerta_soc` . Eles só existem depois da triagem do analista (Kaufman et al., 2012).

O treino limpo fica com **55.074 conexões e 3,30% de anomalias**. O teste já vem limpo: não tem duplicatas nem linhas iguais às do treino.

2. Pré-processamento (requisitos 2 a 5)

Cada escolha venceu uma comparação fold a fold na validação 5×2. Os ganhos são de F1-macro, com o número de folds vencidos entre parênteses.

Etapa	Comparado	Escolha (evidência)
Imputação	mediana/moda, KNN, MissForest, por serviço; categorias por moda, KNN e RF	`service` / `flag` por RF, mediana nos números (+0,0019, 9/10)
Balanceamento	undersampling, SMOTE, `class_weight`, oversampling, SMOTE+Tomek/ENN, ADASYN	nenhum (nenhum passou com limiar ajustado)
Codificação	one-hot, one-hot com raras, target, frequência, ordinal	one-hot, serviços com < 50 linhas juntos (+0,0022, 10/10)
Normalização	Standard, Robust, `log1p`, Quantile, Yeo-Johnson	`log1p` nas contagens + Standard (+0,008 na LR e no KNN, 10/10)
Redução	PCA, variância quase zero, Tomek e ENN	nenhuma (PCA −0,024; filtro −0,009, 0/10)

- **`service` tem alta cardinalidade.**
  - O código ordinal inventa uma ordem entre os serviços (−0,0195 na regressão logística).
  - A frequência junta serviços diferentes de mesmo tamanho (−0,0119).
  - O one-hot com as raras agrupadas também absorve os serviços que só aparecem no teste (`pm_dump`, `printer`).
- **`RobustScaler` falha:** 25 colunas têm IQR = 0.
- **Balanceamento:** com limiar 0,5, o `class_weight` até ajudava a RF (+0,0042). Ajustar o limiar rende mais e não distorce as probabilidades (Elor & Averbuch-Elor, 2022).
- **Redução de dimensionalidade:** o PCA e o filtro de variância apagam as direções raras, como `num_root` diferente de zero, que é onde estão os ataques.

3. Algoritmos e protocolo (requisito 6)

Comparamos três algoritmos: **árvore de decisão**, **random forest** e **rede neural MLP treinada por backpropagation**. Cada resultado da MLP é a média de 3 sementes, porque com uma só o F1 variava ± 0,0066.

O protocolo segue Arp et al. (2022):

- `RepeatedStratifiedKFold` com 5 folds e 2 repetições, os mesmos em todos os experimentos;
- todo o pipeline ajustado só no treino de cada fold;
- o limiar de cada fold escolhido nos outros 4 folds da mesma repetição.

Uma técnica só entra se vencer em pelo menos 8 de 10 folds ou tiver p < 0,10 no teste t corrigido (Bouckaert & Frank, 2004).

4. Tabela comparativa

A random forest é o melhor modelo nas duas versões. A tabela traz o F1-macro médio dos 10 folds.

- **Etapa 2:** pipeline inicial, com limiar 0,5.
- **Final:** limiar ajustado, pré-processamento próprio e hiperparâmetros da busca aleatória.
- **Precisão e revocação:** da classe `anomaly`, sem score.

Modelo	Etapa 2	Final, sem score	Final, com score	Precisão	Revocação
Random forest	0,9756	**0,9846 ± 0,0031**	**0,9909 ± 0,0016**	0,979	0,961
MLP	0,9636	0,9750 ± 0,0049	0,9884 ± 0,0027	0,974	0,930
Árvore de decisão	0,9664	0,9676 ± 0,0058	0,9793 ± 0,0043	0,951	0,924
Classe majoritária	0,4916	0,4916	0,4916	0	0

Sem o score, a random forest vence a MLP nos 10 folds (+0,0096, p = 0,008), e a MLP vence a árvore em 9 de 10 (+0,0074, p = 0,03). O ensemble piora a random forest sem o score (0 de 10 folds) e só empata com ela quando o score entra (p = 0,81), então ficamos com a floresta sozinha.

5. Modelo final e submissão

A random forest final usa `max_features` = 0,2 e `max_depth` = 30 e é treinada nas 55.074 linhas do treino limpo. A chave `USAR_SCORE` do notebook escolhe a versão:

- **sem score:** limiar 0,37, com 1.186 anomalias previstas no teste (4,87%);
- **com score:** limiar 0,475, com 934 anomalias previstas (3,84%).

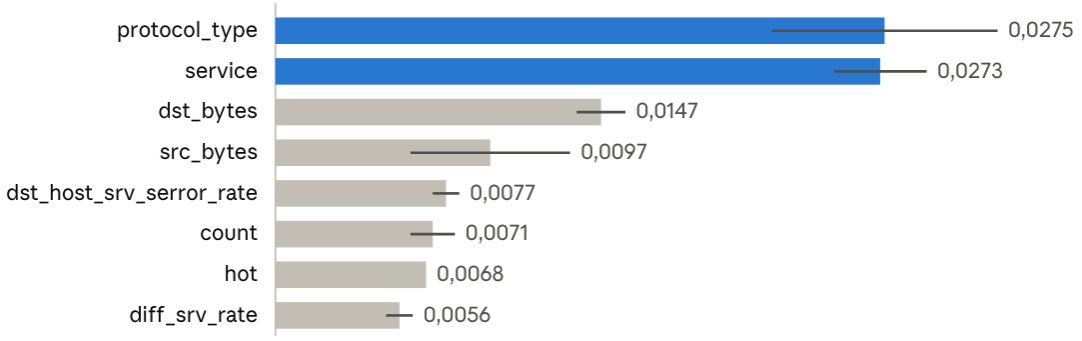
Nos 10 folds, o limiar sem score ficou entre 0,36 e 0,38. O notebook confere o formato do `submissao_<equipe>.csv`: cabeçalho, os 24.333 `id` na ordem do modelo e só os valores `normal` e `anomaly`. Com as sementes fixas, duas execuções geraram o mesmo arquivo.

6. Explicabilidade

O modelo decide pelo tipo de serviço e de protocolo, pelo volume de bytes e pelas taxas de erro de conexão. Seguindo Molnar (2022), usamos três métodos no modelo do fold 0, sobre a validação desse fold, que o modelo não viu, e mais uma árvore substituta. Na permutação, embaralhamos o atributo original, antes do one-hot, então `service` conta como um atributo só.

Embaralhar protocol_type ou service derruba o F1-macro em cerca de 0,027

Queda no F1-macro ao embaralhar cada atributo (validação do fold 0, média de 3 permutações; traço = ± 1 desvio)



etapa4_final.ipynb · versão sem score · 8 dos 40 atributos; 11 não mudam o F1 quando embaralhados

- **Os métodos discordam onde há redundância.** `flag`, `serror_rate` e `srv_serror_rate` medem a mesma coisa: conexões SYN sem resposta. O `flag` é o 1º no SHAP e o 2º na impureza, mas só o 15º na permutação, porque, embaralhado um, a floresta usa os outros.
- **SHAP** (`TreeExplainer`; Lundberg & Lee, 2017), em 600 conexões: empurram para `anomaly` o padrão de uma varredura, com `dst_host_srv_serror_rate` alto, `same_srv_rate` baixo, `flag` = `S0` e poucos bytes.
- **Casos perto do limiar de 0,38:**
  - um ataque pego (0,383) foi decidido por `same_srv_rate` = 0,04 e `service` = `private`;
  - um ataque perdido (0,363) era uma transferência `ftp_data` com cara de normal;
  - um alarme falso (0,410) era uma conexão `private` de 1 byte.
- **Árvore substituta** de profundidade 3: concorda com a floresta em 98,97% das conexões, mas esse número é inflado, porque 96,7% são normais. O F1-macro dela é 0,888, contra 0,988 da floresta. A regra principal é: `dst_host_srv_serror_rate` > 0,95 com `flag` = `S0` ou `SH` indica anomalia.
- **Com o score, ele domina.** Embaralhá-lo tira 0,068 do F1, dez vezes mais do que qualquer outro atributo.

7. Erros, decisão sobre o score e limitações

Os erros que sobram são ataques com cara de conexão normal. Na 1ª repetição, em que cada linha é avaliada uma vez, a versão sem score perde 72 ataques e dá 38 alarmes falsos.

- **83% dos ataques perdidos têm `flag` = `SF`** (conexão completa) e estão em `ftp_data`, `ftp`, `telnet` e `smtp`, os serviços dos ataques de conteúdo (R2L e U2R). Em `ftp_data`, só 70% dos ataques são pegos.
- **A imputação não gera erro:** as linhas com `service` imputado erram menos (0,14%) do que as outras (0,20%).
- **O score legado erra em outro lugar.** Os 38 alarmes falsos têm score abaixo de 0,4, e 67 dos 72 ataques perdidos têm score de 0,6 ou mais. Com o score, os erros caem de 110 para 63.

O `score_ids_legado` não parece vazamento. Investigamos o atributo porque é o mais ligado à classe (AUC de 0,968):

- **Ele não depende do rótulo.** Nos 1.702 pares contraditórios, o lado `normal` e o lado `anomaly` têm sempre o mesmo score. Um atributo derivado da classe seria maior no lado `anomaly`.
- **Ele erra como um detector real.** Só 52% das conexões com score de 0,6 ou mais são ataques.
- **Ele traz informação própria.** Os outros atributos explicam só $R^2 = 0{,}30$ dele.
- **Ele existe no teste,** sem ausentes.

Decisão da equipe: *a preencher*. As evidências apoiam usar o score. A versão sem ele fica no notebook como análise de sensibilidade, porque a origem do score não é documentada.

Limitações:

- Pela distribuição do score, o teste deve ter cerca de 4,2% de ataques. A versão sem score prevê mais do que isso, e a com score, menos.
- Um modelo supervisionado não reconhece ataques que não viu no treino (Sommer & Paxson, 2010).
- A versão com score depende da calibração do IDS legado.

8. Uso de IA e referências

Uso de IA: a equipe usou o Claude (Anthropic) como apoio para três coisas: escrever e depurar o código (células `[IA]`), pesquisar a literatura e revisar o texto. As decisões e a conferência dos resultados foram da equipe.

Referências

- Arp, D. et al. (2022). *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security.
- Bouckaert, R.; Frank, E. (2004). *Evaluating the Replicability of Significance Tests for Comparing Learning Algorithms*. PAKDD.
- Elor, Y.; Averbuch-Elor, H. (2022). *To SMOTE, or not to SMOTE?* arXiv:2201.08528.
- Kaufman, S. et al. (2012). *Leakage in Data Mining*. ACM TKDD.
- Lundberg, S.; Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. NeurIPS.
- Molnar, C. (2022). *Interpretable Machine Learning*, 2ª ed.
- Sommer, R.; Paxson, V. (2010). *Outside the Closed World*. IEEE S&P.
- van den Goorbergh, R. et al. (2022). *The harm of class imbalance corrections for risk prediction models*. JAMIA.